

Approccio Relazionale:

Storie

1. Nel package models/history → ero 3 entità storiche

a) HistorySession

```
@Entity(tableName = "history_sessions")
public class HistorySession implements Serializable {

    @PrimaryKey
    @NonNull
    private String sessionId; // UUID generato alla fine del workout

    private String trainingName; // Es: "Push Day"
    private long startTime;
    private long endTime;
    private long duration;

    public HistorySession() {} // Costruttore vuoto per Room

    public HistorySession(@NonNull String sessionId, String trainingName, long s
        this.sessionId = sessionId;
        this.trainingName = trainingName;
        this.startTime = startTime;
        this.endTime = endTime;
        this.duration = endTime - startTime;
    }

    // Getter e Setter standard...
    public String getSessionId() { return sessionId; }
    public void setSessionId(@NonNull String sessionId) { this.sessionId = sessi
    // ... aggiungi gli altri
}
```

b) HistoryExercise

```
@Entity(tableName = "history_exercises",
foreignKeys = {@ForeignKey(
    entity = HistorySession.class, // Punta alla classe qui sopra
    parentColumns = "sessionId",
    childColumns = "sessionId",
    onDelete = ForeignKey.CASCADE),
    indices = {@Index("sessionId")})
public class HistoryExercise implements Serializable {

    @PrimaryKey
    @NonNull
    private String historyExerciseId;

    private String sessionId; // FK verso la Sessione
    private String exerciseName; // Es: "Panca Piana"
    private int orderIndex; // Per l'ordinamento visivo

    public HistoryExercise() {}

    public HistoryExercise(@NonNull String historyExerciseId, String sessionId,
        this.historyExerciseId = historyExerciseId;
        this.sessionId = sessionId;
        this.exerciseName = exerciseName;
        this.orderIndex = orderIndex;
    }

    // Getter e Setter...
}
```

c) HistorySet

```
@Entity(tableName = "history_sets",
foreignKeys = {@ForeignKey(
    entity = HistoryExercise.class, // Punta all'Esercizio Storico
    parentColumns = "historyExerciseId",
    childColumns = "historyExerciseId",
    onDelete = ForeignKey.CASCADE),
    indices = {@Index("historyExerciseId")})
public class HistorySet implements Serializable {

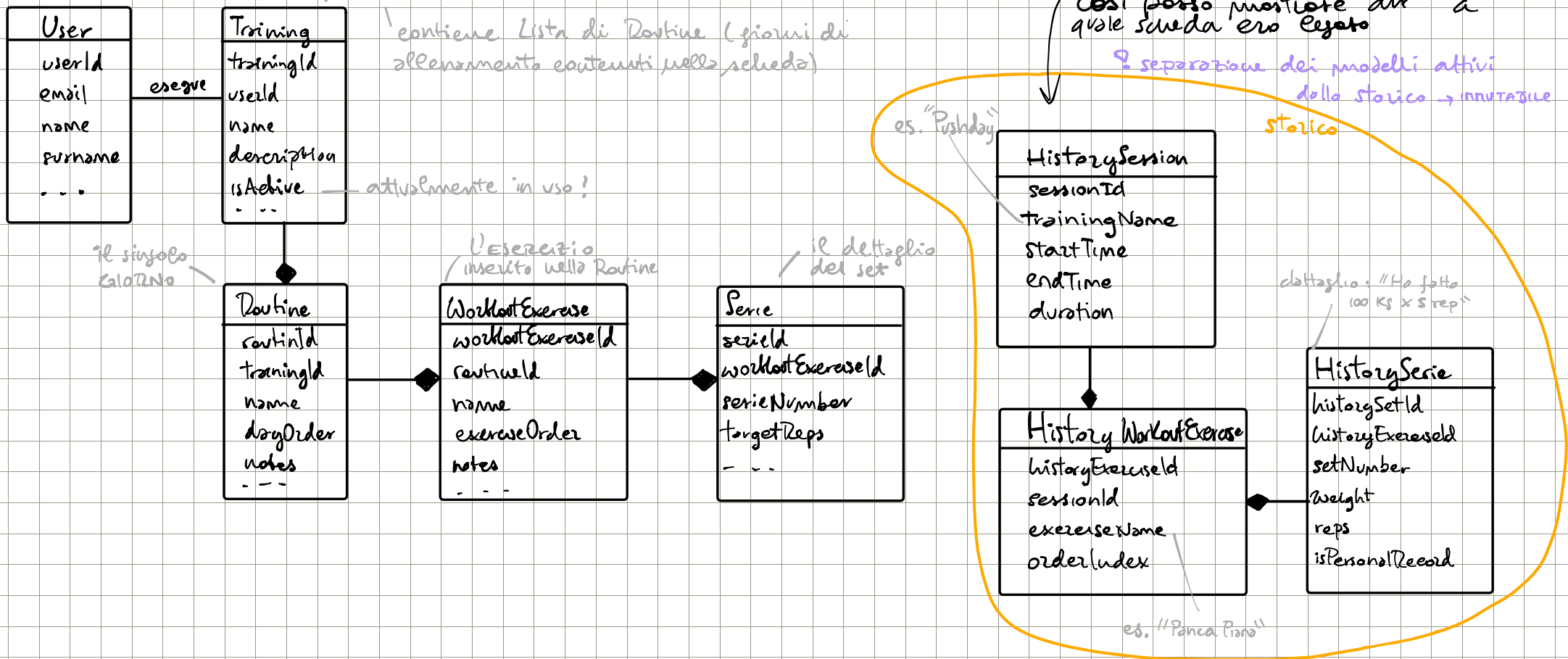
    @PrimaryKey
    @NonNull
    private String historySetId;

    private String historyExerciseId; // FK verso l'Esercizio
    private int setNumber;
    private double weight; // Solo il peso EFFETTIVO
    private int reps; // Solo le rep EFFETTIVE
    private boolean isPr; // Opzionale

    public HistorySet() {}

    public HistorySet(@NonNull String historySetId, String historyExerciseId, in
        this.historySetId = historySetId;
        this.historyExerciseId = historyExerciseId;
        this.setNumber = setNumber;
        this.weight = weight;
        this.reps = reps;
    }

    // Getter e Setter...
}
```



2. Aggiungo classi helper in model/roomModels/helpers

Ⓐ HistorySessionWithExercises

associa una sessione
alla lista dei suoi
esercizi

Ⓑ HistoryExerciseWithSets

associa un esercizio alla lista
dei suoi set

```
public class HistorySessionWithData {  
    @Embedded  
    public HistorySession session; // Usa la classe creata sopra  
  
    @Relation(  
        entity = HistoryExercise.class,  
        parentColumn = "sessionId",  
        entityColumn = "sessionId"  
    )  
    public List<HistoryExerciseWithSets> exercises;  
}
```

```
public class HistoryExerciseWithSets {  
    @Embedded  
    public HistoryExercise exercise; // Usa la classe creata sopra  
  
    @Relation(  
        parentColumn = "historyExerciseId",  
        entityColumn = "historyExerciseId"  
    )  
    public List<HistorySet> sets;  
}
```

3. creo database/HistoryDao (query per salvataggio, lista storico, storico per singolo esercizio)

```
@Dao  
public interface HistoryDao {  
  
    // --- SCRITTURA (atomica) ---  
    @Insert  
    void insertSession(HistorySession session);  
  
    @Insert  
    void insertExercise(HistoryExercise exercise);  
  
    @Insert  
    void insertSet(HistorySet set);  
  
    // --- LETTURA STORICO (Lista) ---  
    // @Transaction serve perché HistorySessionWithData usa @Relation  
    @Transaction  
    @Query("SELECT * FROM history_sessions ORDER BY startTime DESC")  
    List<HistorySessionWithData> getAllHistory();  
  
    // Filtro per singolo esercizio (La query che volevi!)  
    @Transaction  
    @Query("SELECT * FROM history_sessions WHERE sessionId IN " +  
        "(SELECT sessionId FROM history_exercises WHERE exerciseName LIKE '%'" +  
        "ORDER BY startTime DESC)")  
    List<HistorySessionWithData> getHistoryByExercise(String query);  
}
```

mostro storico per
singolo esercizio

4. aggiungo DAO in database/LocalDatabase + 3 entità

```
// 1. IMPORTA I NUOVI MODELLI  
import com.example.pushapp.models.history.HistorySession;  
import com.example.pushapp.models.history.HistoryExercise;  
import com.example.pushapp.models.history.HistorySet;  
  
// ... altri import esistenti ...  
  
@Database(  
    entities = {  
        // ... le tue entità vecchie (User.class, Training.class, ecc.) ...  
        User.class,  
        Training.class,  
        Routine.class,  
        // ... ecc ...  
    },  
    version = 2, // <--- 3. IMPORTANTE: AUMENTA LA VERSIONE (es. da 1 a 2)  
    exportSchema = false  
)  
@TypeConverters({/* ... i tuoi converter ... */})  
public abstract class LocalDatabase extends RoomDatabase {  
  
    // ... i tuoi DAO vecchi ...  
    public abstract UserDao userDao();  
    public abstract TrainingDao trainingDao();  
    // ...  
  
    // 4. AGGIUNGI IL NUOVO DAO  
    public abstract HistoryDao historyDao();  
  
    // ... il resto del codice Singleton ...  
}
```

5. creo repositoryes/HistoryRepository

```
public class HistoryRepository {  
  
    private final HistoryDao historyDao;  
    private final ExecutorService executor;  
    private final FirebaseFirestore firestore;  
    private final String userId;  
  
    // Costruttore utilizzato dal ServiceLocator  
    public HistoryRepository(Application application) {  
        LocalDatabase db = LocalDatabase.getInstance(application);  
        this.historyDao = db.historyDao();  
        this.executor = LocalDatabase.databaseWriteExecutor;  
        this.firestore = FirebaseFirestore.getInstance();  
  
        // Otteniamo l'ID utente per salvare su Firebase nel path giusto  
        if (FirebaseAuth.getInstance().getCurrentUser() != null) {  
            this.userId = FirebaseAuth.getInstance().getCurrentUser().getUid();  
        } else {  
            this.userId = null; // Gestire caso utente non loggato  
        }  
    }  
  
    // Interfaccia generica per le risposte alla UI  
    public interface HistoryCallback<T> {  
        void onSuccess(T data);  
    }  
}
```

```
// ===== SALVATAGGIO (DUAL WRITE: ROOM + FIREBASE) Locale =====
// Salvataggio (DUAL WRITE: ROOM + FIREBASE) Locale
public void saveWorkout(Routine activeRoutine, long startTime, Runnable onSuccess,
    executor.execute() -> {

    // Generiamo gli ID e i timestamp una volta sola
    String sessionId = UUID.randomUUID().toString();
    long endTime = System.currentTimeMillis();
    long duration = endTime - startTime;

    // -----
    // 1. SALVATAGGIO LOCALE (RELAZIONALE - ROOM)
    // Serve per: Grafici veloci, Statistiche, Funzionamento Offline
    // -----
    LocalDatabase.getInstance(null).runInTransaction(() -> {

        // A. Crea la Sessione (Padre)
        HistorySession session = new HistorySession(
            sessionId,
            activeRoutine.getName(),
            startTime,
            endTime
        );
        historyDao.insertSession(session);

        // B. Ciclo Esercizi
        int exerciseOrder = 0;
        List<WorkoutExercise> exercises = activeRoutine.getWorkoutExercises();
        if (exercises != null) {
            for (WorkoutExercise activeEx : exercises) {

                // Controlliamo se c'è almeno un set completato
                boolean hasActivity = false;
                if (activeEx.getSeries() != null) {
                    for (Series s : activeEx.getSeries()) {
                        if (s.isCompleted()) hasActivity = true;
                    }
                }

                if (hasActivity) {
                    // Crea Esercizio Storico
                    String exId = UUID.randomUUID().toString();
                    HistoryExercise historyExercise = new HistoryExercise(
                        exId,
                        sessionId,
                        activeEx.getName(),
                        exerciseOrder++
                    );
                    historyDao.insertExercise(historyExercise);

                    // C. Ciclo Serie (Nipote)
                    int setOrder = 1;
                    for (Series activeSet : activeEx.getSeries()) {
                        if (activeSet.isCompleted()) {
                            HistorySet historySet = new HistorySet(
                                UUID.randomUUID().toString(),
                                exId,
                                setOrder++,
                                activeSet.getActualWeight(), // Salvo
                                activeSet.getActualReps() // Salvo
                            );
                            historyDao.insertSet(historySet);
                        }
                    }
                }
            }
        }
    });
}
```

1) salvataggio dati parziali

"smonterà" Routine e inserisce nelle 3 tabelle dello stories

salvo all'esterno in formato JSON → 1 UNICO DOCUMENTO

```
// 2. SALVATAGGIO CLOUD (DOCUMENTO - FIREBASE)
// Serve per: Backup sicuro, Cambio telefono
// -----
if (user != null) {
    saveToFirebase(activeRoutine, sessionId, startTime, endTime, duration);
}

// Torna al Main Thread per dire "Fatto!" (es. chiudere la schermata)
new Handler(Looper.getMainLooper()).post(onSuccess);
});

// Helper privato per costruire il JSON/map per Firebase
private void saveToFirebase(Routine routine, String sessionId, long start, long end,
    Map<String, Object> sessionMap = new HashMap<>();
    sessionMap.put("sessionId", sessionId);
    sessionMap.put("routineName", routine.getName());
    sessionMap.put("date", start); // Timestamp utile per ordinamento
    sessionMap.put("startTime", start);
    sessionMap.put("endTime", end);
    sessionMap.put("durationMillis", duration);

    List<Map<String, Object>> exercisesList = new ArrayList<>();

    if (routine.getWorkoutExercises() != null) {
        for (WorkoutExercise ex : routine.getWorkoutExercises()) {
            // Mappa per l'esercizio
            Map<String, Object> exMap = new HashMap<>();
            exMap.put("name", ex.getName());

            List<Map<String, Object>> setsList = new ArrayList<>();
            boolean hasCompletedSets = false;

            if (ex.getSeries() != null) {
                for (Series s : ex.getSeries()) {
                    if (s.isCompleted()) {
                        hasCompletedSets = true;
                        Map<String, Object> setMap = new HashMap<>();
                        setMap.put("weight", s.getActualWeight());
                        setMap.put("reps", s.getActualReps());
                        setsList.add(setMap);
                    }
                }
            }

            if (hasCompletedSets) {
                exMap.put("sets", setsList);
                exercisesList.add(exMap);
            }
        }
    }

    sessionMap.put("exercises", exercisesList);

    // Scrittura su Firestore: users/{uid}/history/{sessionId}
    firestore.collection("users")
        .document(userId)
        .collection("history_sessions")
        .document(sessionId)
        .set(sessionMap)
        .addOnFailureListener(e -> Log.e("HistoryRepo", "Errore backup Firebase"));
}
```

caricamento

```
// ===== LETTURA STORICO (LISTA E FILTRI) da locale =====
// -----
public void getAllHistory(HistoryCallback<List<HistorySessionWithData>> callback,
    executor.execute() -> {
    List<HistorySessionWithData> data = historyDao.getAllHistory();
    new Handler(Looper.getMainLooper()).post(() -> callback.onSuccess(data));
}

public void getHistoryForExercise(String query, HistoryCallback<List<HistorySessionWithData>> callback,
    executor.execute() -> {
    // Usa la query modificata (WHERE sessionId IN ...)
    List<HistorySessionWithData> data = historyDao.getHistoryByExercise(query);
    new Handler(Looper.getMainLooper()).post(() -> callback.onSuccess(data));
}

}
```

→ storie

→ storico per esercizio

LA COLLECTION SU FIREBASE

collection / {userId} / history_sessions / {sessionId}

Su Firebase NON conviene dividere in 3 collection! Sarebbe lentissimo e costoso (troppe letture).

Su Firebase, conviene salvare l'allenamento finito come UN UNICO DOCUMENTO (simile all'approccio JSON, ma strutturato come oggetto Firestore).

Esempio di Documento Firestore (history_sessions/session_123):

```
JSON
{
  "sessionId": "abc-123",
  "date": 1738450000,
  "name": "Push Day",
  "exercises": [
    {
      "name": "Panca Piana",
      "sets": [
        { "weight": 100, "reps": 10 },
        { "weight": 100, "reps": 8 }
      ]
    },
    { "name": "Croci", ... }
  ]
}
```

6. Aggiorno WorkoutViewModel : modificherò costruttore + aggiungerò campo repository + metodo finishWorkout

```
public class WorkoutViewModel extends ViewModel {

    // --- REPOSITORIES ---
    private final ExerciseRepository exerciseRepository;
    private final TrainingRepository trainingRepository;
    // 1. NUOVO REPOSITORY PER LO STORICO
    private final HistoryRepository historyRepository;

    // --- VARIABLE PER LA DATA STORICA ---
    // Serve per dire al DB "l'allenamento è avvenuto il 28 Gennaio alle 18:00"
    private long sessionStartTimeMillis = 0L;

    // ... (altri campi esistenti invariati) ...

    // --- LIVE DATA PER NAVIGAZIONE ---
    // Serve per dire al Fragment "Ho finito, chiuditi"
    private final MutableLiveData<boolean> navigateToHome = new MutableLiveData<>();
    public LiveData<boolean> getNavigateToHome() { return navigateToHome; }

    // --- COSTRUTTORE AGGIORNATO ---
    public WorkoutViewModel(TrainingRepository trainingRepository,
        ExerciseRepository exerciseRepository,
        HistoryRepository historyRepository) { // <--- Aggiunto repository
        this.trainingRepository = trainingRepository;
        this.exerciseRepository = exerciseRepository;
        this.historyRepository = historyRepository; // <--- Assegnazione
    }

    // ... (Getters invariati) ...

    // --- LOGICA DI CONTROLLO PRINCIPALE ---
}
```

- Cattura il `sessionStartTime` reale quando inizia l'allenamento.
- Metodo `finishWorkout()` : È snello. Recupera la activeRoutine (piena di dati utente) e la passa al Repository. Al termine, naviga alla Home.

↳ COORDINAZIONE UI - REPOSITORY

```

public void startWorkout(Routine day, Training parentTraining) {
    if (day == null) return;
    this.parentTraining = parentTraining;

    // 2. SALVIAMO IL TIMESTAMP REALE DI ORA (es. 28 Gen 2026, 15:30)
    this.sessionStartTimeMillis = System.currentTimeMillis();

    workoutTitle.setValue(day.getName());
    activeTrainingDay.setValue(day);
    isWorkoutInProgress.setValue(true);
    resetWorkoutTimer();
    startWorkoutTimer();
}

// --- NUOVO METODO FINISH WORKOUT ---
public void finishWorkout() {
    // A. Recuperiamo la Routine attiva (piena di dati inseriti dall'utente)
    Routine activeRoutine = activeTrainingDay.getValue();

    if (activeRoutine == null) return;

    // B. Passiamo la Routine al HistoryRepository
    // Nota: Assicurati che HistoryRepository accetti 'Routine' nel metodo saveWorkout
    historyRepository.saveWorkout(activeRoutine, sessionStartTimeMillis, () -> {

        // C. Callback di Successo (Siamo tornati nel Main Thread)

        // 1. Puliamo lo stato del ViewModel
        resetWorkoutState();

        // 2. Diciamo alla UI di tornare alla Home
        navigateToHome.setValue(true);

    });
}

// ... (Il resto del codice rimane invariato: stopWorkout, timer, logic
}

```

metodo chiamato quando utente clicca
TERMINA Allenamento della UI

7. Aggiungo HistoryRepository al ServiceLocator

```

import android.app.Application; // <--- AGGIUNGI QUESTO IMPORT
import android.content.Context;

import com.example.pushapp.database.LocalDatabase;

public class ServiceLocator {

    private static volatile ServiceLocator instance = null;

    private ServiceLocator() {}

    private volatile LocalDatabase localDatabase = null;

    // ... altri repository esistenti ...
    private volatile TrainingRepository trainingRepository = null;
    private volatile ExerciseRepository exerciseRepository = null;
    private volatile SessionRepository sessionRepository = null;
    private volatile UserRepository userRepository = null;

    // 1. AGGIUNGI LA VARIABILE PER LO STORICO
    private volatile HistoryRepository historyRepository = null;

    // ... DataSource esistenti ...
    // (Se volessi essere super coerente, dovrei creare HistoryLocalDataSource,
    // ma per ora usiamo il repository diretto come definito prima)

    public static ServiceLocator getInstance() {
        if (instance == null) {
            synchronized (ServiceLocator.class) {
                if (instance == null) {
                    instance = new ServiceLocator();
                }
            }
        }
        return instance;
    }

    // --- REPOSITORIES ---

    public synchronized TrainingRepository getTrainingRepository(Context context) {
        if (trainingRepository == null) {
            trainingRepository = new TrainingRepository(
                getTrainingLocalDataSource(context),
                getTrainingRemoteDataSource()
            );
        }
        return trainingRepository;
    }

    // ... getExerciseRepository ...
    // ... getSessionRepository ...
    // ... getUserRepository ...

    // 2. AGGIUNGI IL METODO GETTER PER HISTORY REPOSITORY
    public synchronized HistoryRepository getHistoryRepository(Context context) {
        if (historyRepository == null) {
            // Il costruttore di HistoryRepository vuole (Application application)
            // Usiamo il context per ottenere l'Application
            historyRepository = new HistoryRepository((Application) context.getApplication());
        }
        return historyRepository;
    }

    // ... Il resto del file (DataSources, Database) rimane invariato ...
}

```

garantisce che tutta l'app usi la
stessa istanza del database e del repository

8. Aggiorno ViewModelFactory per recuperare HistoryRepository dal ServiceLocator e passarlo al ViewModel

```
public class ViewModelFactory implements ViewModelProvider.Factory {  
  
    private final Context context;  
  
    public ViewModelFactory(Context context) {  
        this.context = context.getApplicationContext();  
    }  
  
    @NonNull  
    @Override  
    @SuppressWarnings("unchecked")  
    public <T extends ViewModel> T create(@NonNull Class<T> modelClass) {  
  
        if (modelClass.isAssignableFrom(TrainingViewModel.class)) {  
            TrainingRepository trainingRepository = ServiceLocator.getInstance().getTr  
            ExerciseRepository exerciseRepository = ServiceLocator.getInstance().getEx  
            return (T) new TrainingViewModel(trainingRepository, exerciseRepository);  
  
        } else if (modelClass.isAssignableFrom(WorkoutViewModel.class)) {  
            // 1. Recupera i Repo esistenti  
            TrainingRepository trainingRepository = ServiceLocator.getInstance().getTr  
            ExerciseRepository exerciseRepository = ServiceLocator.getInstance().getEx  
  
            // 2. RECUPERA IL NUOVO HISTORY REPOSITORY  
            HistoryRepository historyRepository = ServiceLocator.getInstance().getHisto  
            // 3. Passa tutti e 3 al costruttore del ViewModel  
            return (T) new WorkoutViewModel(trainingRepository, exerciseRepository, his  
  
        } else if (modelClass.isAssignableFrom(UserViewModel.class)) {  
            UserRepository userRepository = ServiceLocator.getInstance().getUserReposi  
            SessionRepository sessionRepository = ServiceLocator.getInstance().getSess  
            return (T) new UserViewModel(userRepository, sessionRepository);  
        }  
  
        throw new IllegalArgumentException("Unknown ViewModel class: " + modelClass.ge  
    }  
}
```

PARTE DI GRAFICI

Grafici → tendino 3 opzioni (x esercizio) ← peso più alto
volumi totale ← formula di peso x ripetizioni (x tutte le serie)
peso medio / 1RM stimato

3. "Peso medio" (Average Weight)

- ✗ SCONSIGLIATO. Matematica ingannevole.
- Il problema: Se l'utente fa un riscaldamento leggero (20kg) e poi una serie pesante (100kg), la media è 60kg. Sembra che si sia allenato "poco", ma in realtà ha spinto 100kg! Il riscaldamento abbassa la media e demotiva.
- Alternativa Migliore: 1RM Stimato (Massimale Teorico). Usa una formula per dire: "Hai fatto 100kg x 5 colpi, quindi teoricamente avresti potuto farne 1 con 116kg". Questo premia lo sforzo, non penalizza il riscaldamento.

① creo models/ GraphPoint

Questa classe non è un'entità del DB, è solo un contenitore per il risultato della query (DTO).

```
Java  
  
package com.example.pushapp.models;  
  
public class GraphPoint {  
    public long date; // Asse X (Timestamp)  
    public float value; // Asse Y (Peso, Volume o 1RM)  
  
    public GraphPoint(long date, float value) {  
        this.date = date;  
        this.value = value;  
    }  
}
```

libreria grafica necessita di X (data) e Y (valore)

GraphPoint salva solo il risultato delle query dello storico

↓

Rappresenta un punto del grafico → mappa il risultato direttamente sulle coordinate

② ⇒ aggiorno HistoryDao

```
@Dao
public interface HistoryDao {

    // -----
    // OPZIONE 1: CARICO MASSIMO (Max Weight)
    // "Il peso singolo più alto sollevato in quel giorno"
    // -----
    @Query("SELECT s.startTime as date, MAX(sets.weight) as value " +
        "FROM history_sessions s " +
        "JOIN history_exercises ex ON s.sessionId = ex.sessionId " +
        "JOIN history_sets sets ON ex.historyExerciseId = sets.historyExerciseId " +
        "WHERE ex.exerciseName = :exerciseName " +
        "GROUP BY s.sessionId " +
        "ORDER BY s.startTime ASC")
    List<GraphPoint> getMaxWeightStats(String exerciseName);

    // -----
    // OPZIONE 2: VOLUME TOTALE (Quello che chiamavi "Peso sessione")
    // "La somma totale di (Peso x Ripetizioni) di tutte le serie"
    // -----
    @Query("SELECT s.startTime as date, SUM(sets.weight * sets.reps) as value " +
        "FROM history_sessions s " +
        "JOIN history_exercises ex ON s.sessionId = ex.sessionId " +
        "JOIN history_sets sets ON ex.historyExerciseId = sets.historyExerciseId " +
        "WHERE ex.exerciseName = :exerciseName " +
        "GROUP BY s.sessionId " +
        "ORDER BY s.startTime ASC")
    List<GraphPoint> getTotalVolumeStats(String exerciseName);

    // -----
    // OPZIONE 3: 1RM STIMATO (Al posto del "Peso Medio")
    // Formula Epley:  $Peso = (1 + Reps/30)$ 
    // Calcoliamo il massimale teorico della serie MIGLIORE di quel giorno
    // -----
    @Query("SELECT s.startTime as date, MAX(sets.weight * (1 + (sets.reps / 30.0))) as value " +
        "FROM history_sessions s " +
        "JOIN history_exercises ex ON s.sessionId = ex.sessionId " +
        "JOIN history_sets sets ON ex.historyExerciseId = sets.historyExerciseId " +
        "WHERE ex.exerciseName = :exerciseName " +
        "GROUP BY s.sessionId " +
        "ORDER BY s.startTime ASC")
    List<GraphPoint> getOneRepMaxStats(String exerciseName);
}
```

③ Aggiorno HistoryRepository

```
// Definisci questo Enum dove preferisci (es. models/enums/StatMetric.java)
public enum StatMetric {
    MAX_WEIGHT,    // Carico Massimo
    TOTAL_VOLUME, // Volume Totale
    ONE_REP_MAX   // 1RM Stimato
}

// Dentro HistoryRepository.java
public void getGraphData(String exerciseName, StatMetric metric, HistoryCallback
    executor.execute(() -> {
        List<GraphPoint> points;

        switch (metric) {
            case TOTAL_VOLUME:
                points = historyDao.getTotalVolumeStats(exerciseName);
                break;
            case ONE_REP_MAX:
                points = historyDao.getOneRepMaxStats(exerciseName);
                break;
            case MAX_WEIGHT:
                points = historyDao.getMaxWeightStats(exerciseName);
                break;
        }

        // Torna al thread principale
        new Handler(Looper.getMainLooper()).post(() -> callback.onSuccess(points));
    });
}
```

• Metodo `getGraphData`: Accetta un Enum (MaxWeight, Volume, 1RM) e decide quale query del DAO chiamare.

Gestione interruzione / chiusura

③ Gestione interruzione allenamento (STOP/CHIUSURA APP)

1. utente vuole terminare l'allenamento
→ TERMINA WORKOUT → CONFERMA? → SI: salvo dati parziali
2. utente chiude app / chiusura inaspettata
→ AVVIO APP: viene rilevato workout in corso ← termina senza salvare + termina e salvo riprendi allenamento

Attributo allenamentoInCorso? NON può essere messo in User o Training
perché?

1. **User** è un'entità che viene sincronizzata con Firebase (contiene dati di profilo). Aggiungere uno stato locale/transitorio come "sto facendo la panca piana ora" creerebbe conflitti di sincronizzazione inutili.
2. **Training** è la definizione del piano (la "Scheda"). Non deve sapere se oggi, alle 18:00, l'hai iniziata.

⇒ soluzione: SharedPreferences

Lo stato "C'è un allenamento in corso?" è uno **Stato dell'Applicazione**, non un dato dell'Utente. Il modo standard Android per gestire questo è usare le SharedPreferences per persistere un "flag" leggero che sopravvive alla chiusura dell'app.

es. Immagina questo scenario realistico:

1. **Telefono**: Stai facendo allenamento. Spunti la casella della prima serie. L'app aggiorna l'oggetto **User** locale aggiungendo "allenamento in corso". Il Sync parte per aggiornare Firebase.
2. **Web/ Tablet**: Nello stesso momento, ti logghi dal PC o da un tablet per cambiare la tua immagine di profilo o correggere il tuo peso corporeo (campo `weight`).
3. **Il Crash**:
 - Il server riceve l'aggiornamento dal Tablet (nuovo peso, nessun allenamento in corso).
 - Il server riceve l'aggiornamento dal Telefono (vecchio peso, con allenamento in corso).

2. Il Problema del "Traffic Jam" (Eccesso di Rete)

L'oggetto **User** è pensato per cambiare raramente (cambi nome, peso o età una volta ogni tanto). Un allenamento in corso cambia ogni secondo:

- Clicchi "Start Timer" → Modifica.
- Spunti un set → Modifica.
- Cambi un carico → Modifica.
- Il timer avanza → Modifica.

Se metti lo stato nell'oggetto **User**, ogni volta che fai una virgola di modifica all'allenamento, il sistema di Sync potrebbe pensare: "Ehi, l'utente è cambiato! Devo inviare tutto l'oggetto **User** a Firebase!". Risultato:

- Consumi batteria e dati inutilmente.
- Rischio di lag nell'app perché il database è sempre impegnato a scrivere.

Stato "allenamento in corso" vive solo sul dispositivo fisico dell'utente

1. Si crea in utils/ una classe dedicata che gestisce la persistenza dello stato dell'allenamento: SessionManager

```
public SessionManager(Context context) {
    this.prefs = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE);
    this.gson = new Gson();
}

// Salva lo stato corrente (Chiamato ogni volta che l'utente modifica un set o mette in pausa)
public void saveSessionState(Routine routine, long startTime) {
    String routineJson = gson.toJson(routine); // Serializziamo l'oggetto complesso
    prefs.edit()
        .putBoolean(KEY_IS_ACTIVE, true)
        .putLong(KEY_START_TIME, startTime)
        .putString(KEY_CURRENT_ROUTINE_JSON, routineJson)
        .apply();
}

// Cancella la sessione (Quando l'utente finisce o annulla)
public void clearSession() {
    prefs.edit().clear().apply();
}

// Controlla se c'era un allenamento in sospeso
public boolean isWorkoutActive() {
    return prefs.getBoolean(KEY_IS_ACTIVE, false);
}

// Recupera la Routine nello stato in cui era (coi checkbox spuntati!)
public Routine getSavesRoutine() {
    String json = prefs.getString(KEY_CURRENT_ROUTINE_JSON, null);
    if (json == null) return null;
    return gson.fromJson(json, Routine.class);
}
```

Ogni volta che un utente completa un set o cambia un peso, il SessionManager salva lo stato attuale della Routine in memoria locale convertendolo temporaneamente in JSON.

```

    public long getStartTime() {
        return prefs.getLong(KEY_START_TIME, 0L);
    }
}

```

2. Gestione scenario 1: terminazione volontaria

HistoryRepository gestisce già i dati parziali in saveWorkout()

→ saveWorkout eside sulle serie:

```

if (activeSet.isCompleted()) { ... salva ... }

```

Quindi, se l'utente ha fatto solo 3 esercizi su 6 e preme "Termina":

1. Il Repository salverà solo le serie completate.
2. Lo storico mostrerà una sessione con meno volume/esercizi.

Il Flusso di Chiamata (Chain of Command)

1. L'Utente preme il bottone "Termina" sul Fragment.
2. Il Fragment chiama: `viewModel.finishWorkout()`.
3. Il ViewModel (dentro `finishWorkout()`):
 - Recupera i dati correnti (`activeRoutine`).
 - Chiama: `historyRepository.saveWorkout(...)`.
 - Nella callback di successo, pulisce il `SessionManager` e naviga alla Home.
4. Il Repository (dentro `saveWorkout()`):
 - Esegue la transazione sul Database (filtra solo i set completati e scrive nelle tabelle).

3. Gestione crash/chiusura app

⇒ SessionManager

↖ No firebase

All'avvio dell'App:

```

Java
// In MainActivity o HomeViewModel
if (sessionManager.isWorkoutActive()) {
    // MOSTRA DIALOG: "Abbiamo rilevato un allenamento interrotto..."
    // Opzione A: RIPRENDI
    // Opzione B: TERMINA E SALVA
    // Opzione C: ELIMINA
}

```

Logica delle Opzioni:

- **Riprendi:**
 1. Recupera la routine: `Routine savedRoutine = sessionManager.getSavedRoutine();`
 2. Recupera il tempo: `long startTime = sessionManager.getStartTime();`
 3. Navighi al `ActiveWorkoutFragment` passando questi dati. Il ViewModel si inizierà con questa routine già compilata invece che crearne una vuota.
- **Termina e Salva (Parziale):**
 1. Recupera `savedRoutine` e `startTime`.
 2. Chiama `historyRepository.saveWorkout(savedRoutine, startTime, ...)` (esattamente come se fosse la fine normale).
 3. Chiama `sessionManager.clearSession()`.
- **Elimina:**
 1. Chiama semplicemente `sessionManager.clearSession()`.

4. Integrazione nel WorkoutViewModel:

Salvataggio continuo → aggiorno stato SessionManager ogni volta che succede qualcosa di importante

Devi aggiungere il campo per il manager, la variabile per il tempo di inizio reale e aggiornare il costruttore per ricevere il manager.

```

Java
public class WorkoutViewModel extends ViewModel {
    // ... altri repository esistenti ...
    private final HistoryRepository historyRepository;

    // --- NUOVO: SESSION MANAGER ---
    private final SessionManager sessionManager;

    // --- NUOVO: TIMESTAMP INIZIO REALE ---
    // Serve per salvare la data/ora storica dell'allenamento (es. 28 Gen 18:00)
    // e passarla al SessionManager per il ripristino.
    private long sessionStartTimeMillis = 0L;

    // --- COSTRUTTORE AGGIORNATO ---
    public WorkoutViewModel(TrainingRepository trainingRepo,
        ExerciseRepository exerciseRepo,
        HistoryRepository historyRepo,
        SessionManager sessionManager) { // <--- Aggiungilo!
        this.trainingRepository = trainingRepo;
        this.exerciseRepository = exerciseRepo;
        this.historyRepository = historyRepo;
        this.sessionManager = sessionManager; // <--- Assegnazione
    }

    // ...
}

```

(AGGIORNARE FACTORY)

inizio allenamento:

```

Java
public void startWorkout(Routine day, Training parentTraining) {
    if (day == null) return;
    this.parentTraining = parentTraining;

    // --- NUOVO: CATTURA ORA DI INIZIO ---
    // Se stiamo riprendendo un crash, questo valore dovrebbe arrivarci da fuori
    // ma per un nuovo allenamento lo settiamo a ORA.
    if (this.sessionStartTimeMillis == 0) {
        this.sessionStartTimeMillis = System.currentTimeMillis();
    }

    workoutTitle.setValue(day.getName());
    activeTrainingDay.setValue(day);
    isWorkoutInProgress.setValue(true);
    resetWorkoutTimer();
    startWorkoutTimer();

    // --- NUOVO: PRIMO SALVATAGGIO (AUTOSAVE) ---
    // "SessionManager, segna che abbiamo iniziato!"
    sessionManager.saveSessionState(day, sessionStartTimeMillis);
}

```

modifica checkbox:

```

public void toggleSetCompleted(int exercisePosition, int setPosition, int restFir
Routine currentDay = activeTrainingDay.getValue();
if (currentDay == null) return;

// ... (tutta la logica esistente che cambia isCompleted) ...

// Aggiorna UI
activeTrainingDay.setValue(currentDay);

// --- NUOVO: SYNC DOPO IL CLICK ---
// Salviamo lo stato aggiornato (con la spunta messa/tolta)
sessionManager.saveSessionState(currentDay, sessionStartTimeMillis);
}

```

modifica peso/ reps:

```

/*
 * Chiamato quando l'utente modifica il testo (Peso o Reps) e perde il focus
 * o preme invio sulla tastiera.
 */
public void updateSetData(int exercisePosition, int setPosition, double actualWe
Routine currentDay = activeTrainingDay.getValue();

// Controlli di sicurezza standard
if (currentDay == null || currentDay.getWorkoutExercises() == null) return;
List<WorkoutExercise> exercises = currentDay.getWorkoutExercises();

if (exercisePosition < exercises.size()) {
    WorkoutExercise ex = exercises.get(exercisePosition);
    if (ex.getSeries() != null && setPosition < ex.getSeries().size()) {
        Serie serie = ex.getSeries().get(setPosition);

        // 1. Aggiorna i dati in memoria
        serie.setActualWeight(actualWeight);
        serie.setActualReps(actualReps);

        // 2. --- NUOVO: SYNC IMMEDIATO ---
        // Se l'app crasha ora, al riavvio ritroviamo il peso modificato
        sessionManager.saveSessionState(currentDay, sessionStartTimeMillis);
    }
}
}

```


fine allenamento → salvo + pulisco backup

```
Java

public void finishWorkout() {
    Routine activeRoutine = activeTrainingDay.getValue();
    if (activeRoutine == null) return;

    // Chiamata al Repository (Salvataggio Permanente)
    historyRepository.saveWorkout(activeRoutine, sessionStartTimeMillis, () -> {

        // --- CALLBACK DI SUCCESSO ---

        // 1. --- NUOVO: PULIZIA BACKUP ---
        // Il DB è al sicuro, possiamo cancellare il file temporaneo
        sessionManager.clearSession();

        // 2. Resetta ViewModel
        resetWorkoutState();

        // 3. Naviga via
        navigateToHome.setValue(true);
    });
}
```